LINKED STRUCTURES

SIXTH-WEEK ASSIGNMENT FOR THE DATA STRUCTURE

Lecturer :

Andi Iwan Nurhidayat, S.Kom., M.T.

Presented by :

Aisyah Currents

25091397108 / 2025I

INFORMATIC MANAGEMENT MAJOR

FACULTY OF VOCATION

UNIVERSITAS NEGERI SURABAYA

YEAR 2026

PRACTICUM

1. Integer values are implemented and manipulated at the hardware-level, allowing for fast operations. But the hardware does not supported unlimited integer values. For example, when using a 32-bit architecture, the integers are limited to the range -2,147,483,648 through 2,147,483,647. If you use a 64-bit architecture, this range is increased to the range -9,223,372,036,854,775,808 through 9,223,372,036,854,775,807. But what if we need more than 19 digits to represent an integer value?

   In order to provide platform-independent integers and to support integers larger than 19 digits, Python implements its integer type in software. That means the storage and all of the operations that can be performed on the values are handled by executable instructions in the program and not by the hardware. Learning to implement integer values in software offers a good example of the need to provide efficient implementations. We define the Big Integer ADT below that can be used to store and manipulate integer values of any size, just like Python's built-in `int` type.

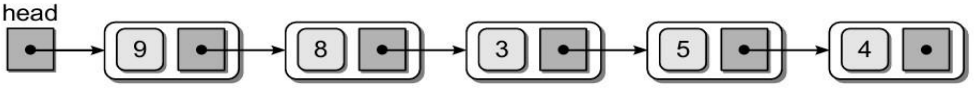
   - `BigInteger( initValue = "0" )`: Creates a new big integer that is initialized to the integer value specified by the given string.
   - *`toString`*`()`: Returns a string representation of the big integer.
   - *`comparable`*`( other )`: Compares this big integer to the `other` big integer to determine their logical ordering. This comparison can be done using any of the logical operators: `<, <=, >, >=, ==, !=`.
   - *`arithmetic`*`( rhsInt )`: Returns a new `BigInteger` object that is the result of performing one of the arithmetic operations on the `self` and `rhsInt` big integers. Any of the following operations can be performed:

     `+  -  *  //  %  **`

   - *`bitwise-ops`*`( rhsInt )`: Returns a new `BigInteger` object that is the result of performing one of the bitwise operators on the `self` and `rhsInt` big integers. Any of the following operations can be performed:

     `|  &  ^  <<  >>`

   (a) Implement the Big Integer ADT using a singly linked list in which each digit of the integer value is stored in a separate node. The nodes should be ordered from the least-significant digit to the largest. For example, the linked list below represents the integer value 45,839:

   (b) Implement the Big Integer ADT using a Python list for storing the individual digits of an integer.

2. Modify your implementation of the Big Integer ADT from the previous question by adding the assignment combo operators that can be performed on the `self` and `rhsInt` big integers. Allow for any of the following operations to be performed:

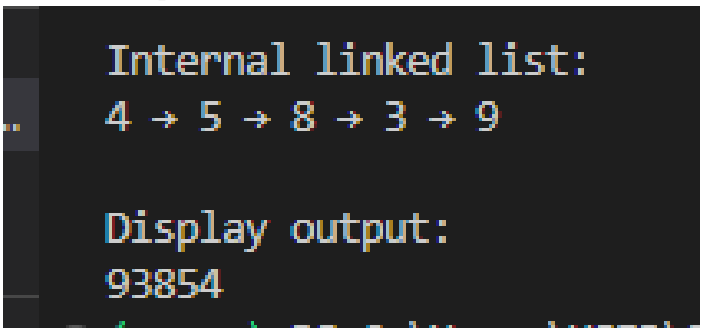
```
+=     -=     *=     //=     %=      **=
<<=    >>=    |=     &=      ^=
```

The Answers:

1. (a) The Source Code

```
1.     class Node:
2.      def __init__(self, digit):
3.          self.digit = digit
4.          self.next = None
5.
6.  class BigIntegerLL:
7.      def __init__(self, number):
8.          self.head = None
9.          for digit in str(number)[::-1]:  # reverse order
10.             self.insert(int(digit))
11.
12.     def insert(self, digit):
13.         new_node = Node(digit)
14.         new_node.next = self.head
15.         self.head = new_node
16.
17.     def print_list(self):
18.         current = self.head
19.         result = []
20.         while current:
21.             result.append(str(current.digit))
22.             current = current.next
23.         return " → ".join(result)
24.
25.     def display(self):
26.         current = self.head
27.         result = ""
28.         while current:
29.             result = str(current.digit) + result
30.             current = current.next
31.         return result
32.
33.# Run program
34.num = BigIntegerLL(45839)
35.
36.print("\nInternal linked list:")
37.print(num.print_list())
38.
39.print("\nDisplay output:")
40.print(num.display())
```

The Output



The Explaination

This program implements a Big Integer Abstract Data Type (ADT) using a singly linked list. The Node class is used to represent each digit of the number, where each node stores a single digit and a reference to the next node. The BigIntegerLL class manages the linked list structure. When an object is initialized, the input number is first converted into a string and reversed using slicing ([::-1]) so that the least significant digit is processed first. Each digit is then inserted into the linked list using the insert() method, which adds a new node at the head of the list. As a result, the linked list stores digits in reverse order internally. The print_list() method traverses the linked list and displays the stored digits in sequence using arrows, showing the internal representation. Meanwhile, the display() method reconstructs the original number by traversing the list and combining the digits in the correct order. This approach allows the program to represent large integers efficiently by storing each digit separately rather than relying on built-in integer limits.

(b) The Source Code

```
1. class BigIntegerList:
2.     def __init__(self, number):
3.         self.digits = [int(d) for d in str(number)][::-1]
4.
5.     def display(self):
6.         return ''.join(map(str, self.digits[::-1]))
7.
8. # Run program
9. num = BigIntegerList(45839)
10.
11.print("Input number:", 45839)
12.print("\nList representation (reversed):")
13.print(num.digits)
14.
15.print("\nDisplay output:")
16.print(num.display())
```

The Output

```
Input number: 45839

List representation (reversed):
[9, 3, 8, 5, 4]

Display output:
45839
```

The Explaination

This program implements a Big Integer Abstract Data Type (ADT) using a Python list. The BigIntegerList class stores each digit of a number as an element in a list. When the object is initialized, the input number is first converted into a string, then each digit is extracted and stored in a list in reverse order using slicing ([::-1]). This means the least significant digit is stored at the beginning of the list, which simplifies certain arithmetic operations. The display() method reconstructs the original number by reversing the list back to its correct order and joining the digits into a single string. In the main program, an instance is created with the number 45839, and the program prints the original input, the internal list representation, and the reconstructed number. This approach is simpler and more efficient than a linked list because Python lists provide built-in functionality for indexing and dynamic storage.

2. The Source Code

```
class BigInteger:
    def __init__(self, number):
        self.value = int(number)

    # Arithmetic operators
    def __iadd__(self, other):
        self.value += other.value
        return self

    def __isub__(self, other):
        self.value -= other.value
        return self

    def __imul__(self, other):
        self.value *= other.value
        return self

    def __ifloordiv__(self, other):
        self.value //= other.value
        return self

    def __imod__(self, other):
        self.value %= other.value
        return self

    def __ipow__(self, other):
```

```
        self.value **= other.value
        return self

    # Bitwise operators
    def __ilshift__(self, other):
        self.value <<= other.value
        return self

    def __irshift__(self, other):
        self.value >>= other.value
        return self

    def __ior__(self, other):
        self.value |= other.value
        return self

    def __iand__(self, other):
        self.value &= other.value
        return self

    def __ixor__(self, other):
        self.value ^= other.value
        return self

    def __str__(self):
        return str(self.value)


# MAIN PROGRAM
A = BigInteger(1000)
B = BigInteger(200)

print("Initial values:")
print("A =", A)
print("B =", B)

A += B
print("\nAfter A += B:")
print("A =", A)

A -= B
print("\nAfter A -= B:")
print("A =", A)

A *= B
print("\nAfter A *= B:")
print("A =", A)

A //= B
print("\nAfter A //= B:")
print("A =", A)
```

```
80.
81.A %= B
82.print("\nAfter A %= B:")
83.print("A =", A)
84.
85.A **= BigInteger(2)
86.print("\nAfter A **= 2:")
87.print("A =", A)
88.
89.A <<= BigInteger(1)
90.print("\nAfter A <<= 1:")
91.print("A =", A)
92.
93.A >>= BigInteger(1)
94.print("\nAfter A >>= 1:")
95.print("A =", A)
96.
97.A |= B
98.print("\nAfter A |= B:")
99.print("A =", A)
100.
101.      A &= B
102.      print("\nAfter A &= B:")
103.      print("A =", A)
104.
105.      A ^= B
106.      print("\nAfter A ^= B:")
107.      print("A =", A)
```

The Output

```
Initial values:
A = 1000
B = 200

After A += B:
A = 1200

After A -= B:
A = 1000

After A *= B:
A = 200000

After A //= B:
A = 1000

After A %= B:
A = 0

After A **= 2:
A = 0
```

```
After A <<= 1:
A = 0
After A >>= 1:
A = 0

After A |= B:
A = 200

After A &= B:
A = 200

After A ^= B:
A = 0
```

The Explaination

This program implements a Big Integer Abstract Data Type (ADT) using operator overloading in Python. The BigInteger class stores integer values and allows them to behave like built-in integers by overloading assignment combination operators. Arithmetic operations such as addition (+=), subtraction (-=), multiplication (*=), floor division (//=), modulus (%=), and exponentiation (**=) are implemented through special methods like __iadd__, __isub__, __imul__, __ifloordiv__, __imod__, and __ipow__. In addition, bitwise operations such as left shift (<<=), right shift (>>=), OR (|=), AND (&=), and XOR (^=) are supported using their corresponding methods. Each operation updates the current object's value and returns the modified object, enabling in-place computation similar to standard Python integers. The __str__ method is defined to allow the object to be printed in a readable format. Overall, this implementation demonstrates how operator overloading can be used to extend the functionality of user-defined classes, making them intuitive and consistent with built-in data types.